



Факультет Систем Управления и Робототехники

РОБОТ С ДИФФЕРЕНЦИАЛЬНЫМ ПРИВОДОМ

Аннотация – В работе исследовано движение робота с дифференциальным приводом при использовании двух способов управления: P-регулятора и тангенциального регулятора. Для обоих законов управления построены модели в Simulink, проведены эксперименты на реальном роботе EV3 и выполнено сравнение экспериментальных и модельных траекторий. Отдельно рассмотрены одиночные переходы в заданные точки и последовательное движение по квадратной траектории.

Ключевые слова: дифференциальный привод; одометрия; P-регулятор; тангенциальный регулятор; Simulink; EV3

Выполнили

Мостафа Эхаб Ахмед Абделвахаб Мохамед¹,
Ануфриев Андрей Сергеевич²

¹ 503341

² 465029

Проверил

Овчаров Алексей Олегович

1 Постановка задачи

Цель работы — построить математическую модель робота с дифференциальным приводом, реализовать два закона управления движением к цели, сравнить экспериментальные данные с моделированием в Simulink и оценить, насколько хорошо модель описывает поведение реального робота.

Для достижения цели в работе были поставлены следующие задачи:

1. изучить кинематическую модель робота с дифференциальным приводом;
2. реализовать одометрию для вычисления координат робота по данным энкодеров;
3. реализовать Р-регулятор и тангенциальный регулятор для движения к целевой точке;
4. построить обе схемы управления в Simulink;
5. провести эксперименты при движении из точки $(0, 0)$ в точки $(1, 0)$, $(0, 1)$, $(-1, 0)$ и $(0, -1)$;
6. провести эксперимент с движением по квадратной траектории;
7. обработать данные и построить графики для каждого регулятора отдельно и для их общего сравнения;
8. проанализировать расхождения между моделью и экспериментом и сделать выводы.

2 Теоретическая часть

2.1 Математическая модель робота

Положение робота на плоскости задаётся координатами x , y и углом ориентации θ . Кинематическая модель робота с дифференциальным приводом имеет вид

$$\dot{x} = v \cos \theta, \quad (1)$$

$$\dot{y} = v \sin \theta, \quad (2)$$

$$\dot{\theta} = \omega, \quad (3)$$

где v — линейная скорость центра робота, а ω — угловая скорость.

Если за малый промежуток времени левое и правое колёса повернулись на углы $\Delta\psi_l$ и $\Delta\psi_r$, то путь центра робота и изменение угла можно вычислить по формулам

$$\Delta L = \frac{(\Delta\psi_l + \Delta\psi_r)r}{2}, \quad (4)$$

$$\Delta\theta = \frac{(\Delta\psi_r - \Delta\psi_l)r}{B}, \quad (5)$$

где r — радиус колеса, B — база робота.

В работе использовались параметры робота EV3: $r = 29$ мм и $B = 120$ мм. Эти же значения применялись в программе и в модели Simulink.

2.2 Одометрия

Одометрия позволяет оценивать текущее положение робота по приращениям углов колёс. На каждом шаге вычисляются приращения координат

$$\Delta x = \Delta L \cos \theta, \quad (6)$$

$$\Delta y = \Delta L \sin \theta, \quad (7)$$

$$\Delta \theta = \frac{(\Delta \psi_r - \Delta \psi_l)r}{B}, \quad (8)$$

после чего координаты обновляются:

$$x_i = x_{i-1} + \Delta x, \quad (9)$$

$$y_i = y_{i-1} + \Delta y, \quad (10)$$

$$\theta_i = \theta_{i-1} + \Delta \theta. \quad (11)$$

В модели такой подход работает хорошо, но в реальном эксперименте ошибка постепенно накапливается. Причина не только в проскальзывании, но и в неидеальности моторов, дискретности энкодеров, несимметрии колёс и в том, что сама модель является упрощённой.

2.3 Р-регулятор

Для движения в точку (x_g, y_g) вводятся расстояние до цели

$$\rho = \sqrt{(x_g - x)^2 + (y_g - y)^2}, \quad (12)$$

азимут на цель

$$\psi = \text{atan2}(y_g - y, x_g - x), \quad (13)$$

и курсовая ошибка

$$\alpha = \psi - \theta. \quad (14)$$

В Р-регуляторе используются выражения

$$U_s = K_s \rho, \quad (15)$$

$$U_r = K_r \alpha, \quad (16)$$

а сигналы на моторы задаются как

$$U_{left} = U_s - U_r, \quad (17)$$

$$U_{right} = U_s + U_r. \quad (18)$$

Такой регулятор прост по структуре, но форма траектории зависит от соотношения двух управляющих частей: поступательной $U_s = K_s \rho$ и поворотной $U_r = K_r \alpha$. Если в начале движения угол ошибки α велик, то робот одновременно получает команду и на движение вперёд, и на поворот. Из-за этого он не доворачивается на месте, а начинает идти по дуге. Чем больше отношение $K_s \rho / K_r \alpha$ в начальный момент, тем сильнее робот уходит вперёд до полного доворота, и тем шире получается траектория.

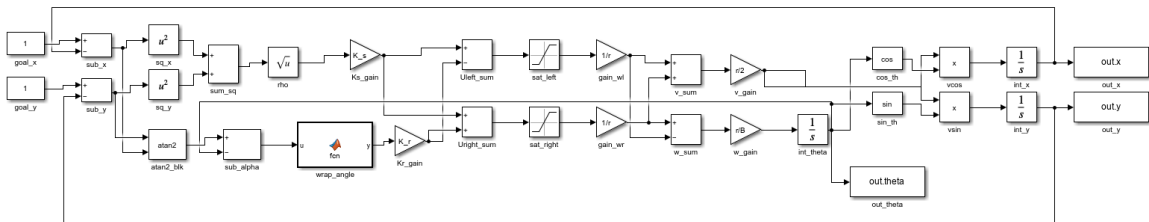


Рис. 1: Схема Р-регулятора в Simulink

2.4 Тангенциальный регулятор

В тангенциальном регуляторе линейная и угловая составляющие задаются формулами

$$v_{goal} = K_1 \rho \cos(\alpha), \quad (19)$$

$$\omega_{goal} = K_1 \cos(\alpha) \sin(\alpha) + K_2 \alpha. \quad (20)$$

Сигналы на моторы задаются как

$$U_{left} = v_{goal} - \omega_{goal}, \quad (21)$$

$$U_{right} = v_{goal} + \omega_{goal}. \quad (22)$$

У тангенциального регулятора форма траектории определяется иначе. Линейная скорость содержит множитель $\cos(\alpha)$, то есть при большой ошибке по углу движение вперёд автоматически уменьшается. Если $\alpha \approx \pm \frac{\pi}{2}$, то $\cos(\alpha)$ близок к нулю, и робот почти не едет вперёд, а в основном доворачивается. Поэтому траектория получается более компактной, а боковой уход в начале движения меньше, чем у Р-регулятора.

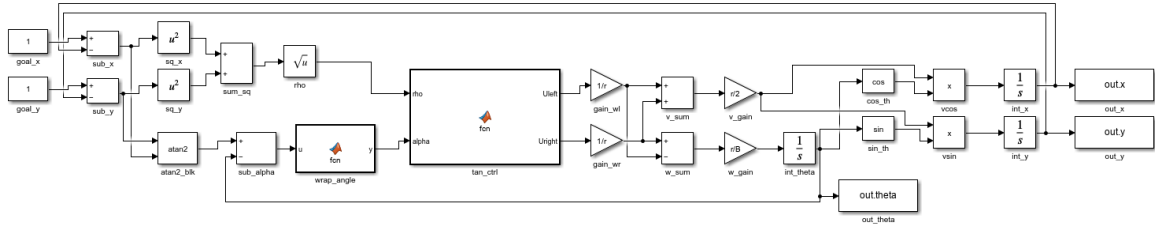


Рис. 2: Схема тангенциального регулятора в Simulink

3 Результаты экспериментов

В этой части сначала отдельно рассматриваются результаты для Р-регулятора, затем для тангенциального регулятора, а после этого приводится их общее сравнение. Такой порядок позволяет сначала оценить поведение каждого регулятора самого по себе, а потом сравнить их между собой.

3.1 Р-регулятор

Одиночные переходы

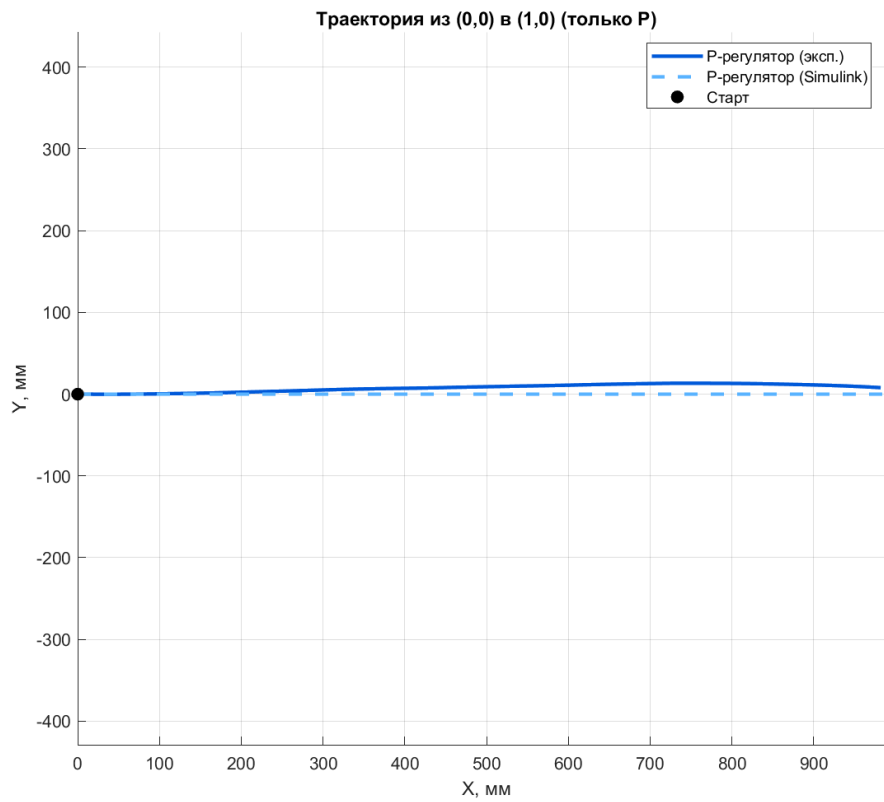


Рис. 3: Р-регулятор: движение из (0,0) в (1,0)

При движении в точку (1,0) траектория близка к прямой, так как начальная ошибка по углу мала. В этом случае сигнал U_r невелик, и робот в основном движется вперёд. Поэтому эксперимент и модель совпадают здесь лучше всего.

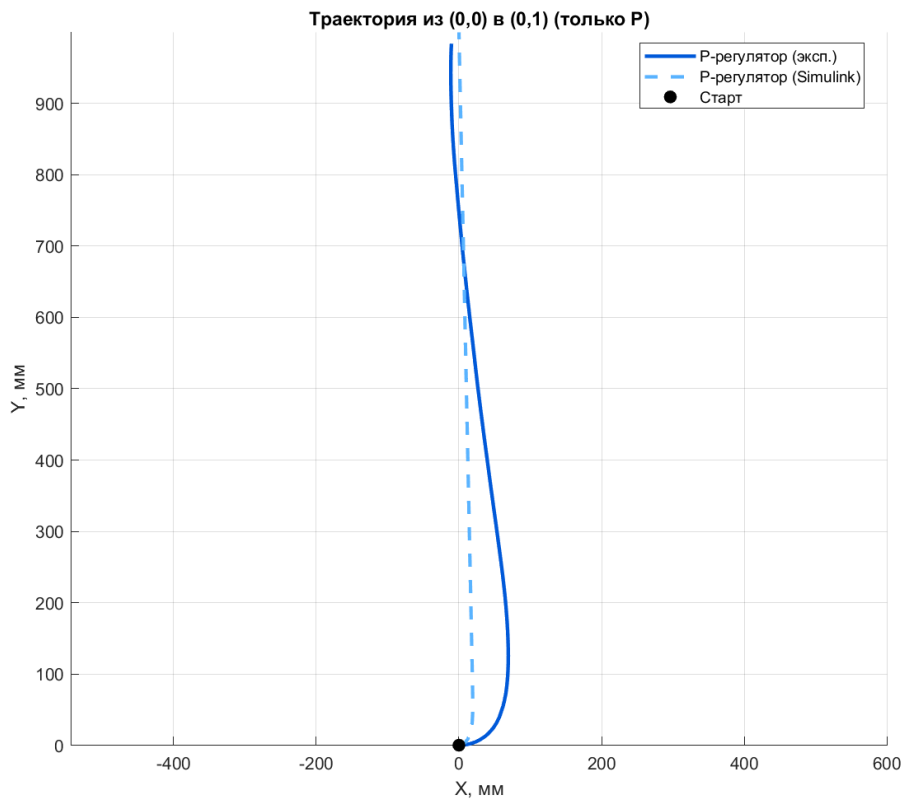


Рис. 4: P-регулятор: движение из $(0, 0)$ в $(0, 1)$

При движении в точку $(0, 1)$ сначала возникает заметная дуга. Это связано с тем, что робот одновременно едет вперёд и поворачивает: при большой α сигнал U_r велик, но U_s тоже остаётся ненулевым. Поэтому робот не доворачивается на месте, а входит в траекторию по кривой.

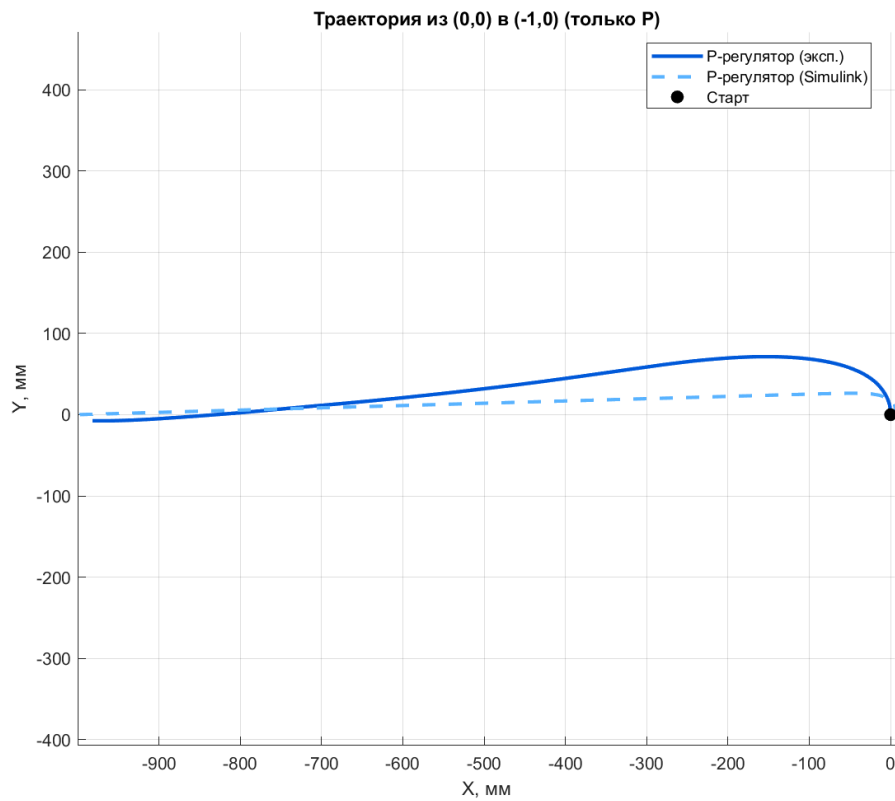


Рис. 5: P-регулятор: движение из $(0,0)$ в $(-1,0)$

Для точки $(-1,0)$ различие формы траектории становится наибольшим. Здесь начальная угловая ошибка близка к 180° , и робот долго перестраивает курс. В результате траектория получается широкой, а влияние несимметрии моторов и ошибки одометрии усиливается.

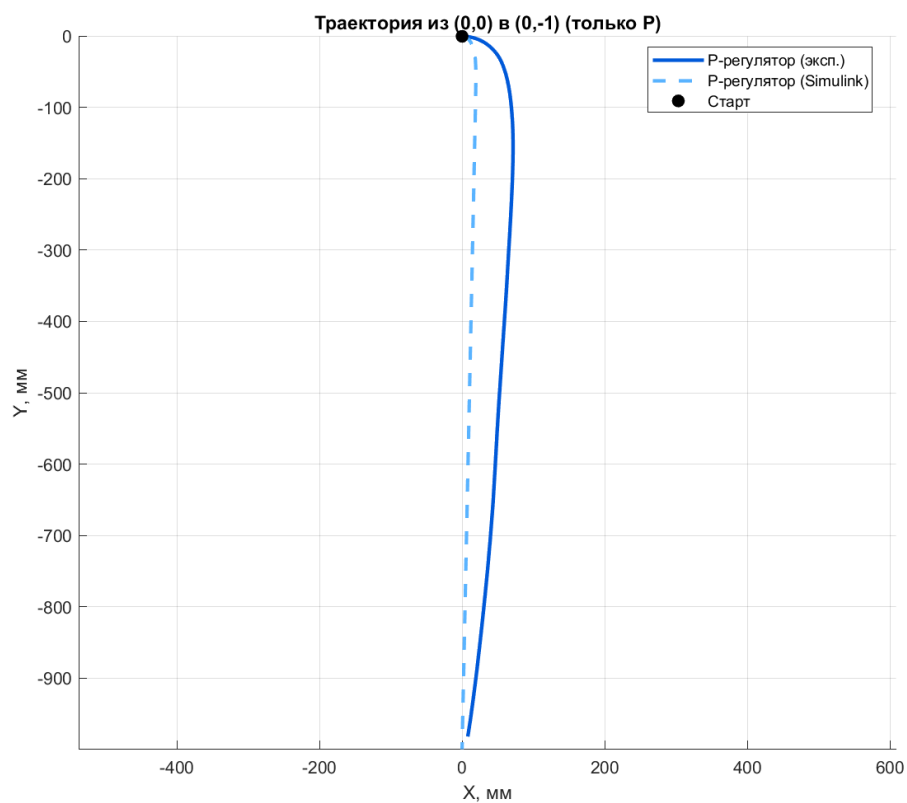


Рис. 6: P-регулятор: движение из $(0, 0)$ в $(0, -1)$

Для перехода в точку $(0, -1)$ наблюдается тот же эффект: траектория сначала изгибается, а затем выпрямляется по мере уменьшения α . Значит, для P-регулятора характерен широкий вход в траекторию при больших начальных углах.

Сравнение траекторий Р-регулятора

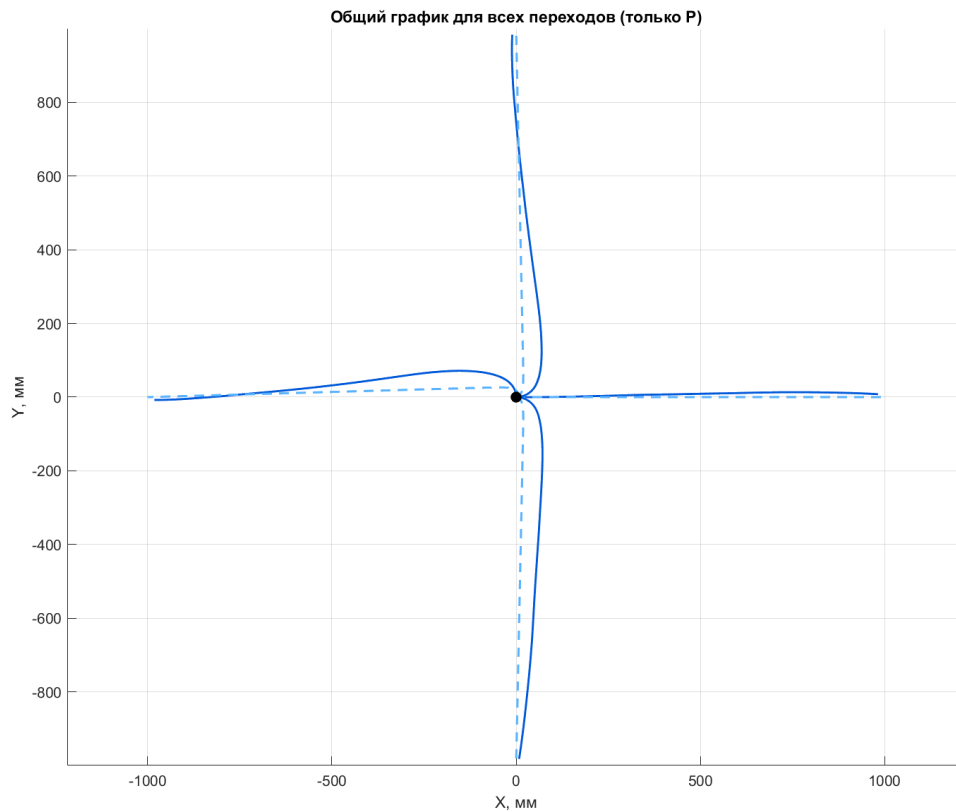


Рис. 7: Все одиночные переходы для Р-регулятора

Если сравнивать именно форму траекторий, то у Р-регулятора хорошо видно общее свойство: чем больше начальный угол между направлением робота и целью, тем шире дуга. То есть различие между переходами определяется не только положением цели, но и величиной α в начальный момент.

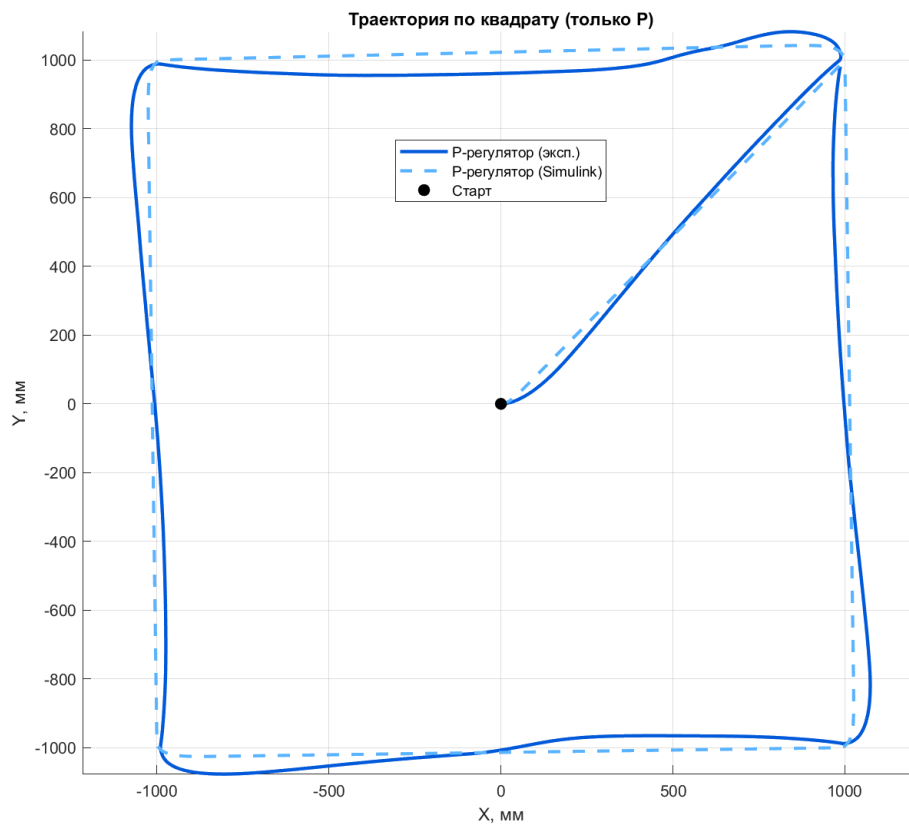


Рис. 8: Движение по квадрату для P-регулятора

На квадратной траектории это проявляется особенно ясно. После каждого поворота робот входит в новую сторону не по прямой, а по сглаженной дуге. Поэтому вершины квадрата оказываются частично “срезанными”, а после нескольких сторон появляется заметное смещение всей траектории.

3.2 Тангенциальный регулятор

Одиночные переходы

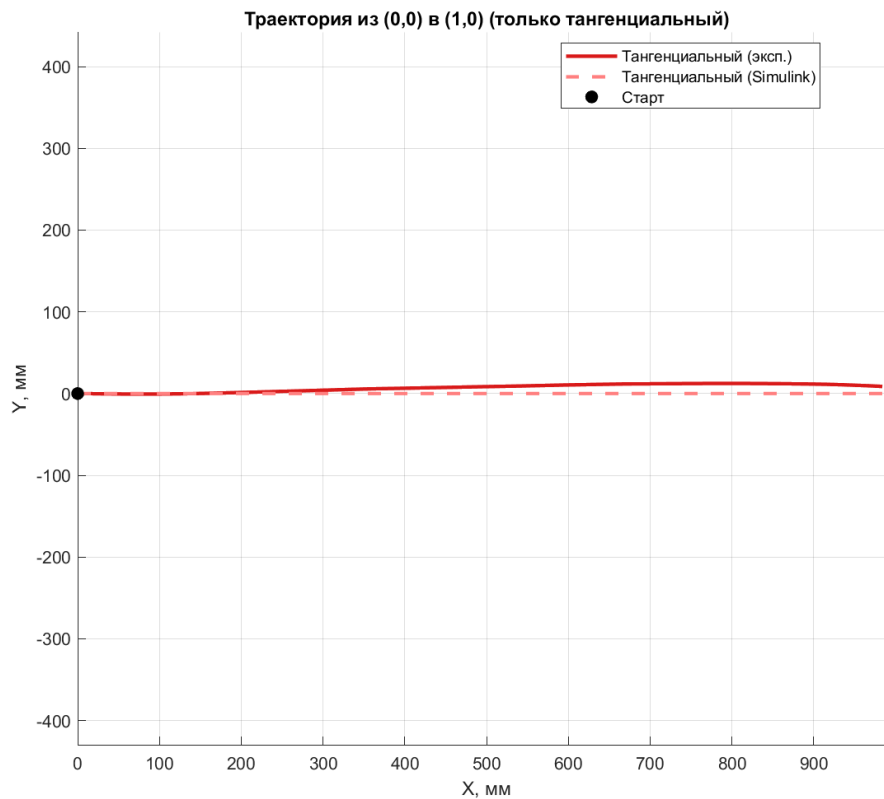


Рис. 9: Тангенциальный регулятор: движение из (0,0) в (1,0)

При движении в точку (1,0) траектория также близка к прямой. Здесь угол ошибки мал, поэтому множитель $\cos(\alpha)$ почти не меняет линейную скорость, и оба регулятора ведут себя похоже.

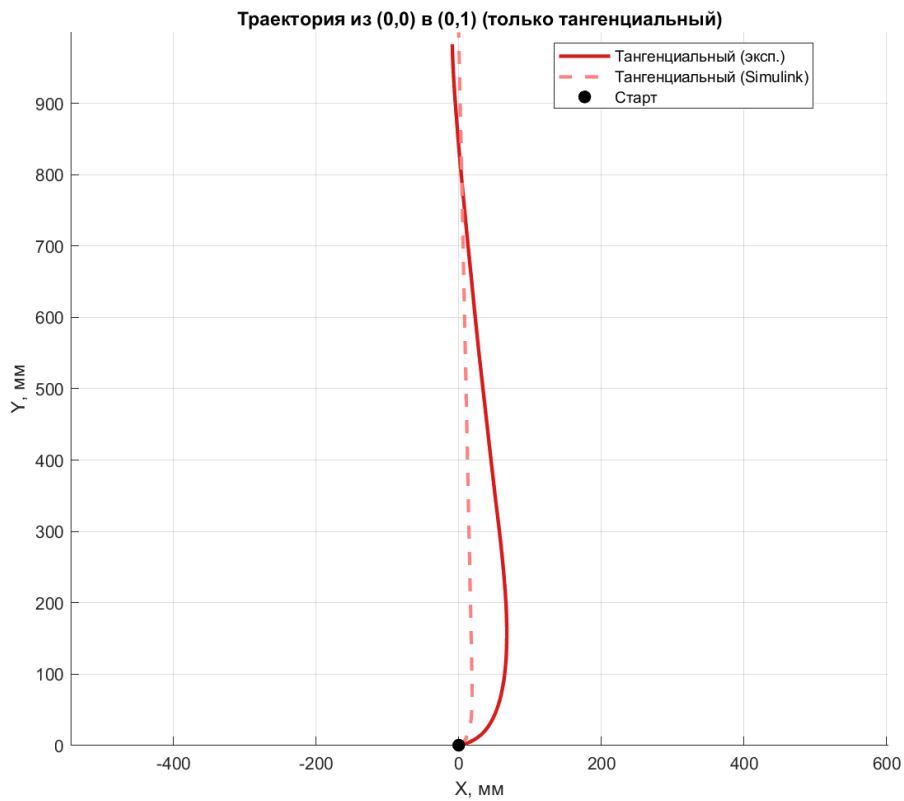


Рис. 10: Тангенциальный регулятор: движение из $(0,0)$ в $(0,1)$

При движении в точку $(0,1)$ траектория уже заметно компактнее, чем у Р-регулятора. Причина в том, что при большом α линейная скорость уменьшается из-за множителя $\cos(\alpha)$, и робот сначала доворачивается, а потом активнее идёт вперёд. Поэтому боковой уход меньше.

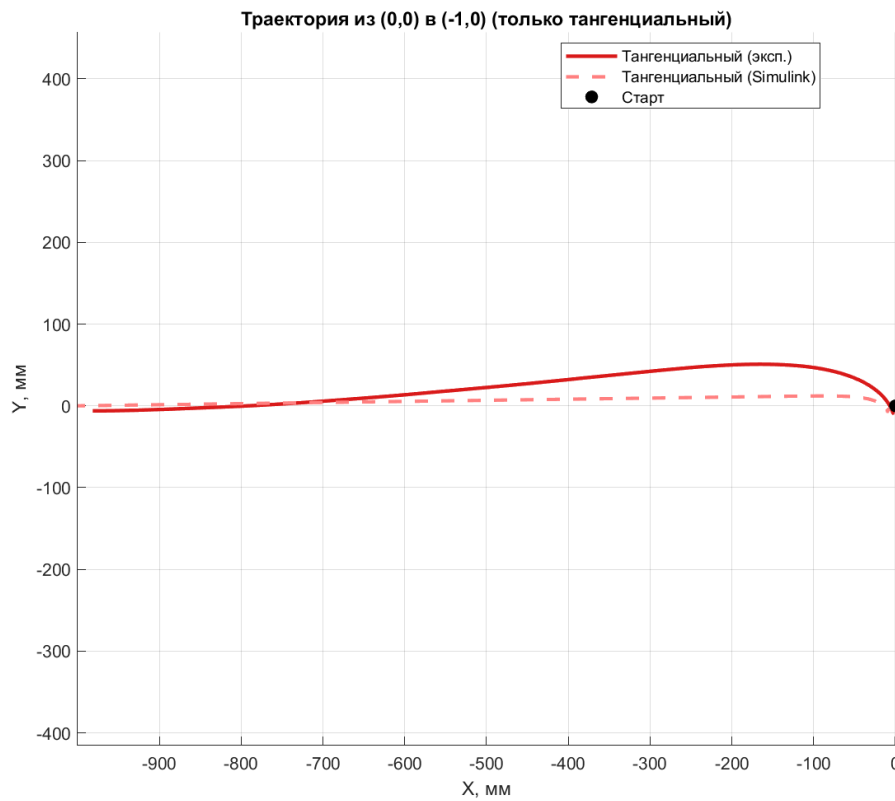


Рис. 11: Тангенциальный регулятор: движение из $(0,0)$ в $(-1,0)$

Для точки $(-1,0)$ преимущество особенно заметно. Там, где Р-регулятор строит широкую дугу, тангенциальный регулятор даёт более собранную траекторию. Это связано с тем, что при большой угловой ошибке движение вперёд подавляется, и робот меньше уходит в сторону.

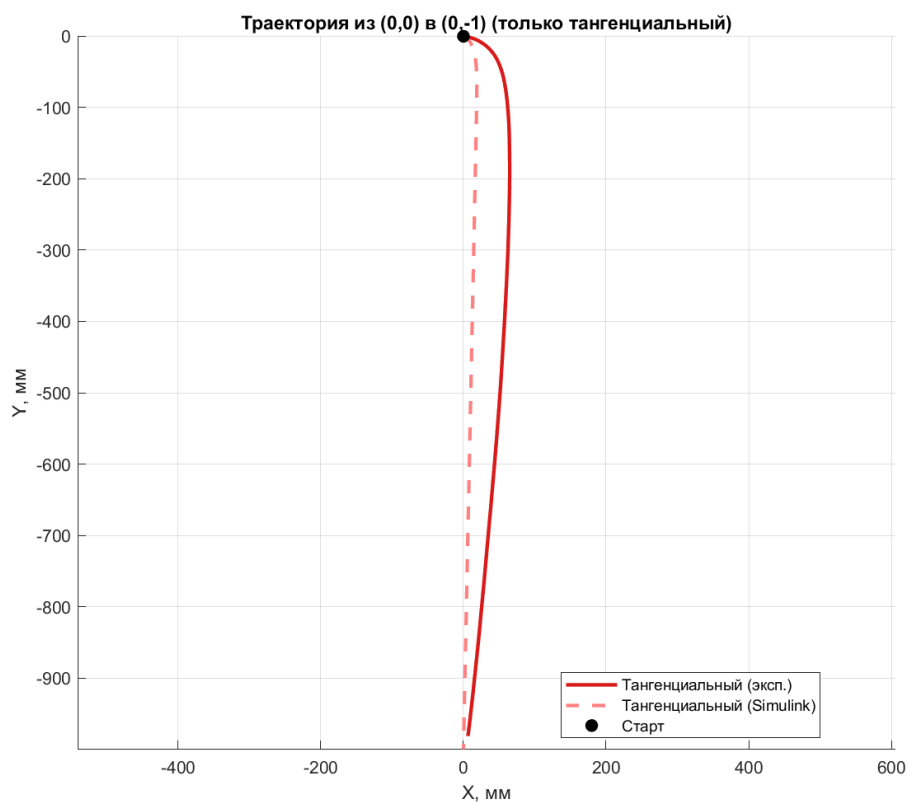


Рис. 12: Тангенциальный регулятор: движение из $(0, 0)$ в $(0, -1)$

При переходе в точку $(0, -1)$ наблюдается тот же эффект. Траектория остаётся более узкой, а вход в направление к цели происходит быстрее. Это значит, что тангенциальный регулятор лучше работает в случаях большого начального углового рассогласования.

Сравнение траекторий тангенциального регулятора

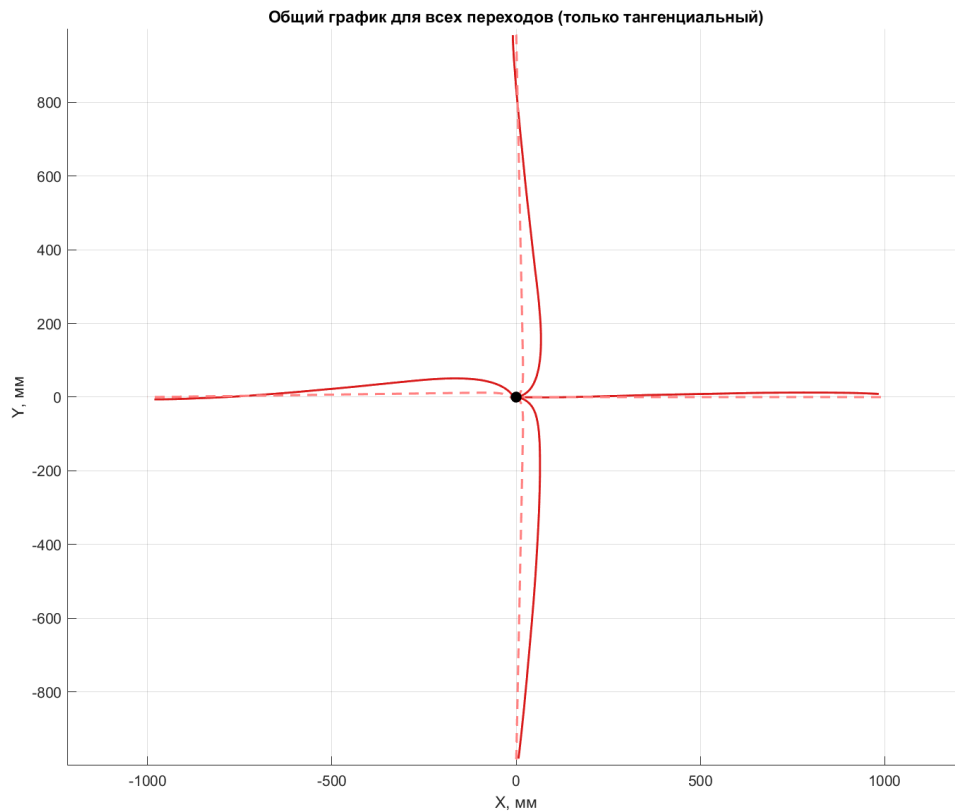


Рис. 13: Все одиночные переходы для тангенциального регулятора

Если сравнивать именно форму траекторий, то у тангенциального регулятора они менее растянуты вбок и ближе друг к другу по характеру. Это означает, что регулятор слабее зависит от начального угла и лучше сохраняет структуру движения при разных положениях цели.

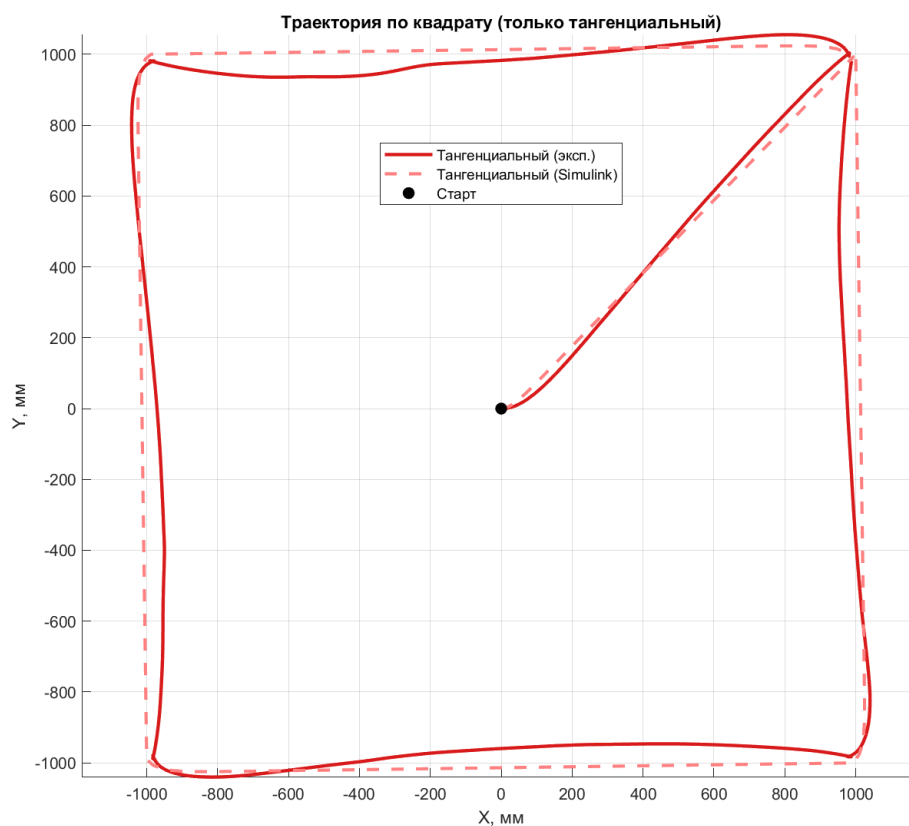


Рис. 14: Движение по квадрату для тангенциального регулятора

На квадратной траектории повороты между сторонами получаются короче по переходному участку. Вершины тоже не идеальны, но они выражены лучше, чем у Р-регулятора. Значит, при последовательных манёврах тангенциальный закон управления лучше сохраняет форму заданной траектории.

3.3 Общее сравнение регуляторов

Сравнение для отдельных переходов

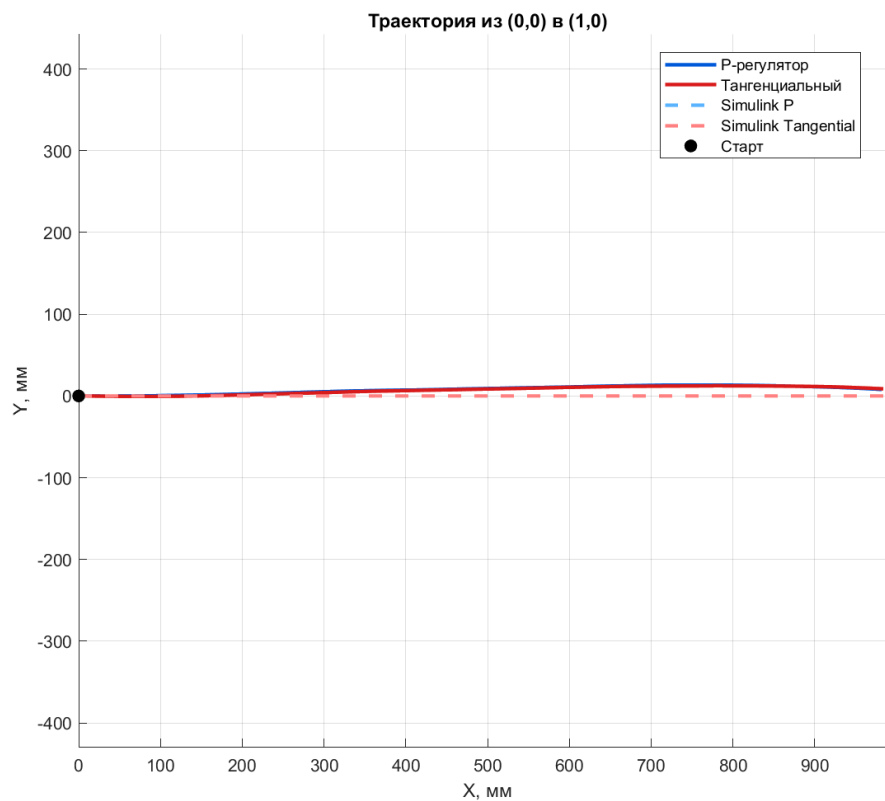


Рис. 15: Сравнение двух регуляторов при движении в точку (1,0)

В случае движения в точку (1,0) оба регулятора дают близкий результат. Здесь задача самая простая, поэтому преимущества более сложного закона управления выражены слабо.

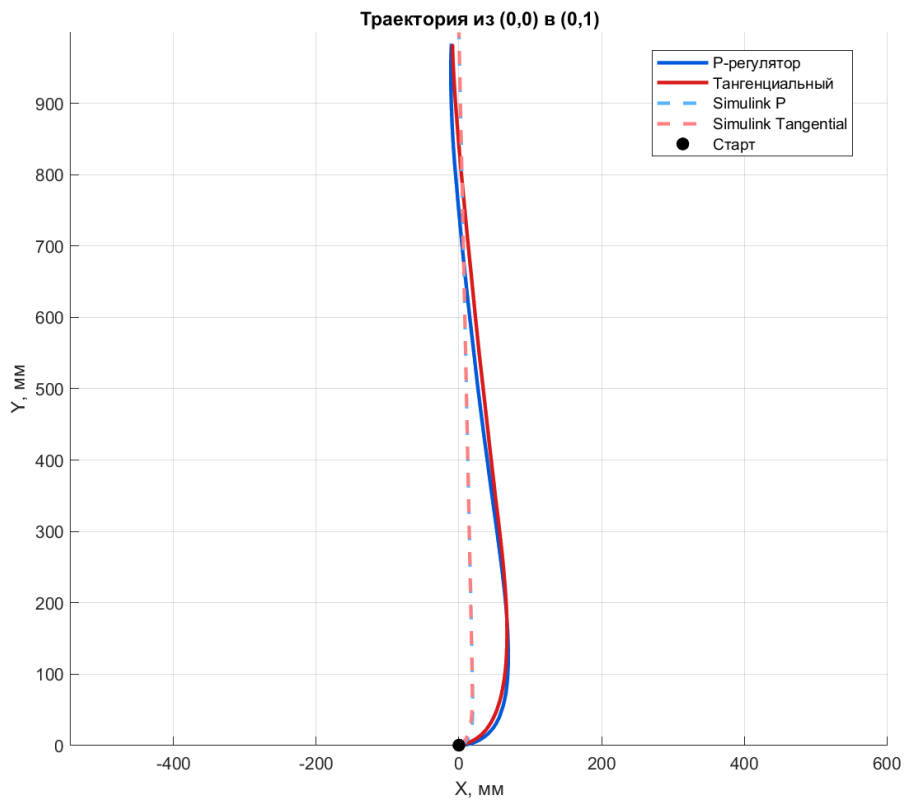


Рис. 16: Сравнение двух регуляторов при движении в точку $(0,1)$

При переходе в точку $(0,1)$ разница уже заметнее. Тангенциальный регулятор строит более узкую траекторию и меньше уходит в сторону на этапе выхода к цели.

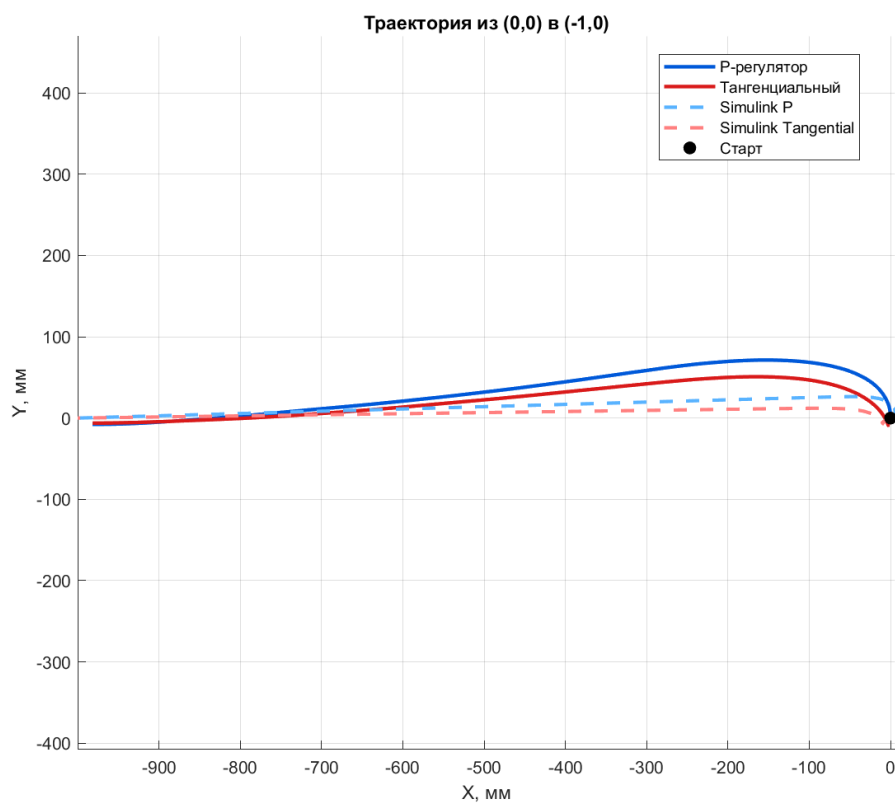


Рис. 17: Сравнение двух регуляторов при движении в точку $(-1, 0)$

Для цели позади начального положения преимущество тангенциального регулятора выражено наиболее сильно. Он лучше справляется с большими начальными угловыми ошибками и реже формирует слишком широкую дугу.

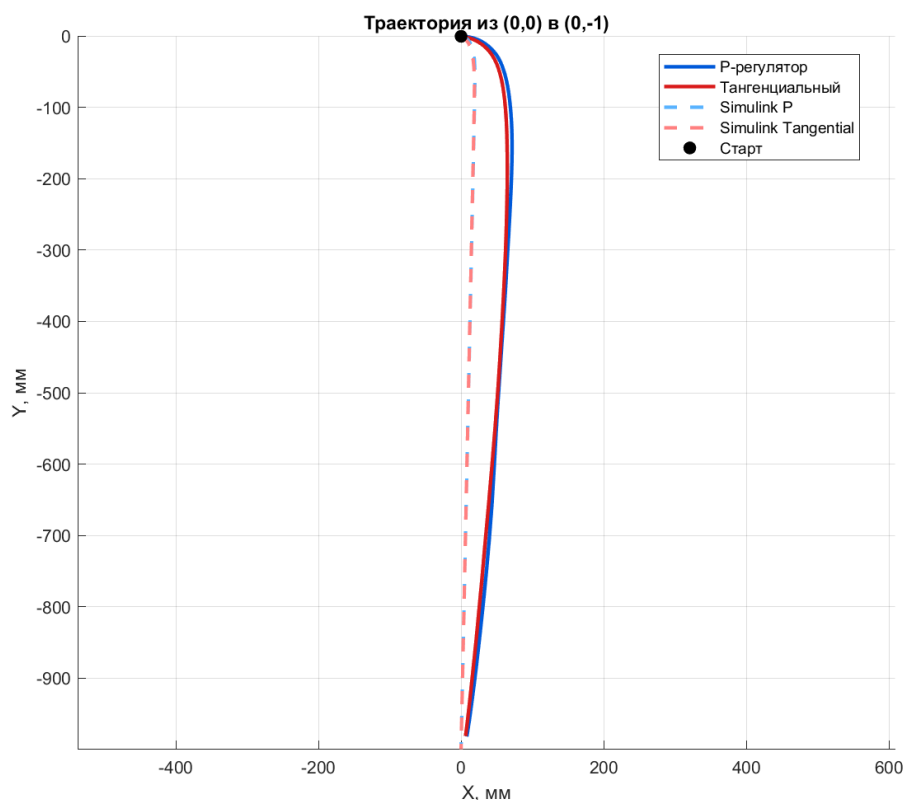


Рис. 18: Сравнение двух регуляторов при движении в точку $(0, -1)$

Случай $(0, -1)$ даёт такой же вывод. Оба регулятора решают задачу, но тангенциальный делает это более аккуратно по форме траектории.

Сравнение формы траекторий

Если сравнивать регуляторы именно по форме траекторий, то главное различие состоит в следующем: у P-регулятора траектория чаще начинается широкой дугой, а у тангенциального регулятора — более коротким участком доворота с меньшим боковым уходом. Это особенно хорошо видно для целей $(0, 1)$, $(-1, 0)$ и $(0, -1)$, где начальная ошибка по углу велика.

Математически это объясняется структурой законов управления. У P-регулятора поступательное движение определяется как $U_s = K_s \rho$, то есть зависит только от расстояния до цели и не уменьшается само по себе при большой угловой ошибке. Поэтому робот продолжает ехать вперёд даже тогда, когда ещё не довёрнут к цели. У тангенциального регулятора линейная скорость содержит множитель $\cos(\alpha)$, поэтому при больших значениях α движение вперёд уменьшается. Именно из-за этого его траектория обычно уже и ближе к направлению на цель.

Для последовательного движения по квадрату это различие становится ещё заметнее. У P-регулятора каждая новая сторона начинается с более длинного переходного дугового участка, поэтому форма квадрата постепенно искажается. У тангенциального регулятора переход между сторонами короче, поэтому общая траектория лучше сохраняет геометрию маршрута.

Сравнение всех одиночных переходов

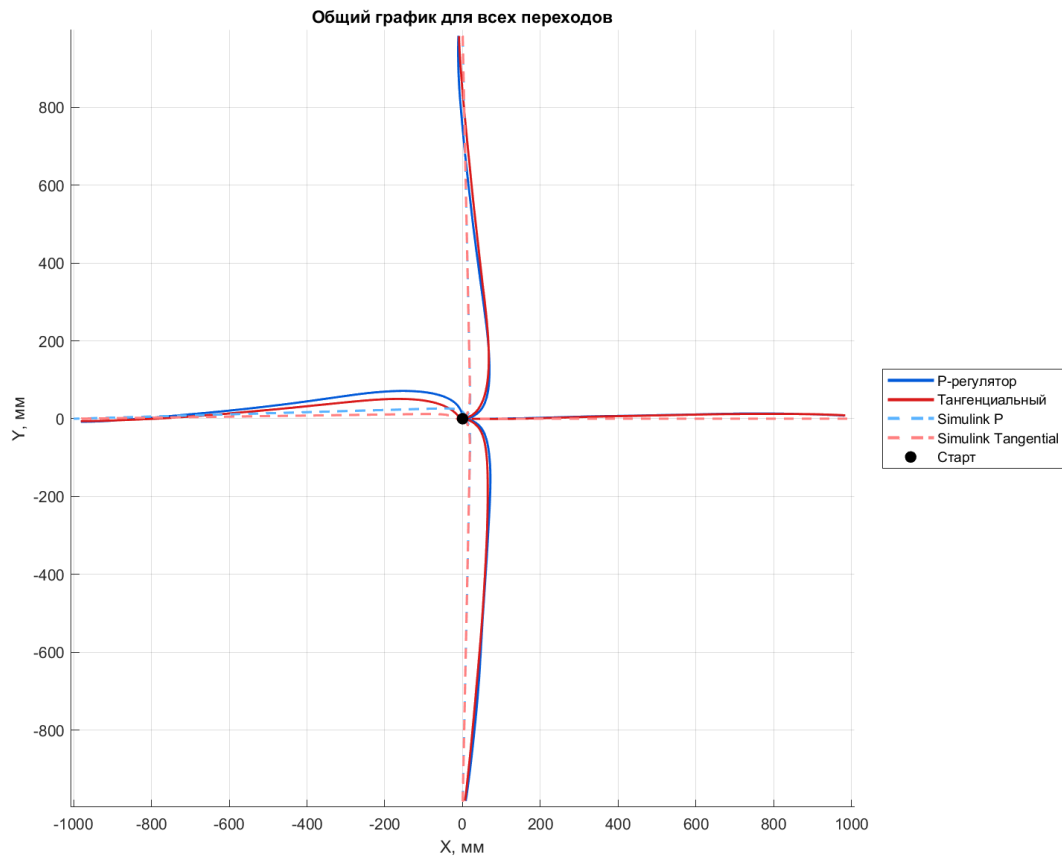


Рис. 19: Общий график всех одиночных переходов

На этом графике видно, что в простых переходах разница между регуляторами невелика, а в переходах с сильным разворотом траектории тангенциального регулятора оказываются компактнее. Значит, его преимущество проявляется прежде всего в более сложной геометрии движения.

Сравнение движения по квадрату

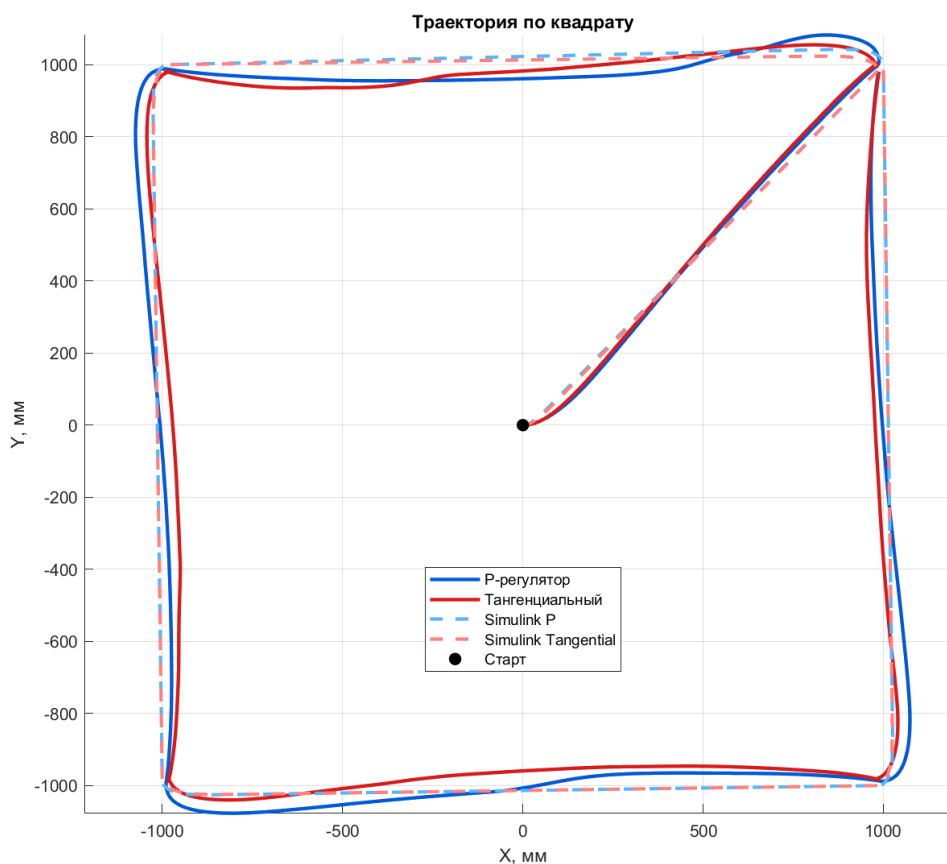


Рис. 20: Сравнение двух регуляторов при движении по квадрату

Движение по квадрату является самым показательным испытанием, потому что здесь одновременно присутствуют прямолинейные участки, развороты и накопление ошибки от шага к шагу. На этом графике видно, что тангенциальный регулятор в среднем лучше сохраняет форму траектории, а P-регулятор сильнее искажает квадрат после нескольких последовательных поворотов.

4 Анализ различий между моделью и экспериментом

Расхождения между моделью и экспериментом связаны не с одной причиной, а с наложением нескольких факторов.

Во-первых, модель является кинематической. В ней предполагается, что заданный сигнал сразу приводит к нужной скорости колёс. В реальном роботе моторы имеют инерцию и не выходят мгновенно на требуемый режим. Поэтому в начале движения и при поворотах экспериментальная траектория отличается от расчётной.

Во-вторых, в модели левое и правое колёса считаются полностью одинаковыми, а база робота — постоянной. В реальной конструкции даже небольшая асимметрия колёс, различие трения или смещение центра масс приводит к систематическому отклонению. На длинной траектории эта ошибка накапливается.

В-третьих, сравнение выполняется по данным одометрии, а не по внешнему измерению координат. Значит, часть расхождения связана не только с движением робота, но и с ошиб-

кой оценки его положения. Особенно сильно это проявляется на квадратной траектории, где ошибка каждой стороны переходит на следующую.

В-четвёртых, различие определяется и самим видом регулятора. У Р-регулятора движение вперёд не ослабляется при большой угловой ошибке, поэтому траектория получается более широкой. У тангенциального регулятора линейная скорость уменьшается через множитель $\cos(\alpha)$, поэтому робот сначала лучше ориентируется на цель и только потом активнее движется вперёд.

Следовательно, различие между моделью и экспериментом объясняется совместным действием упрощённой кинематики, ошибки одометрии, неидеальности моторов и различной структуры законов управления. Именно поэтому тангенциальный регулятор в эксперименте обычно даёт более компактные траектории, хотя полного совпадения с моделью всё равно не возникает.

5 Заключение

В работе были построены и исследованы два способа управления роботом с дифференциальным приводом: Р-регулятор и тангенциальный регулятор. Для обоих случаев были собраны модели в Simulink, проведены эксперименты на роботе EV3 и построены графики одиночных переходов и движения по квадрату.

Главный результат состоит в том, что оба регулятора решают задачу вывода робота в целевую точку, но формируют разные траектории. Р-регулятор даёт более широкие дуги при больших начальных углах, потому что движение вперёд сохраняется даже при большой ошибке ориентации. Тангенциальный регулятор формирует более компактные траектории, так как его линейная скорость уменьшается при росте угловой ошибки.

Сравнение с экспериментом показало, что кинематическая модель правильно передаёт общий характер движения, но не описывает его полностью. Основные трудности были связаны с накоплением ошибки одометрии, неидеальностью моторов и тем, что реальный робот реагирует на управление не так, как идеализированная модель. Практическая ценность работы состоит в том, что удалось не только реализовать управление и моделирование, но и понять, как структура регулятора влияет на форму траектории реального мобильного робота.

А Листинг программы на Python

```
1 from ev3dev2.motor import LargeMotor, OUTPUT_A, OUTPUT_B # pyright: ignore[
    reportMissingImports]
2 from math import pi, cos, sin, radians, atan2, copysign
3 from time import time
4 MAX_SPEED = 50.0
5 class Motor:
6     def __init__(self, motor):
7         self._motor = motor
8     def set_power(self, power):
9         self._motor.run_direct(duty_cycle_sp=min(MAX_SPEED, max(-MAX_SPEED,
10             power)))
11     def get_degree(self):
12         return self._motor.position
13 class Robot:
14     def __init__(self, left_motor, right_motor):
15         self._left_motor = Motor(left_motor)
16         self._right_motor = Motor(right_motor)
17         self._prev_left_degree = self._left_motor.get_degree()
```

```

17     self._prev_right_degree = self._right_motor.get_degree()
18     def set_powers(self, left_power, right_power):
19         # print("power", left_power, right_power)
20         self._left_motor.set_power(left_power)
21         self._right_motor.set_power(right_power)
22     def get_delta_degrees(self):
23         left_degree = self._left_motor.get_degree()
24         right_degree = self._right_motor.get_degree()
25         delta_left_degree = left_degree - self._prev_left_degree
26         delta_right_degree = right_degree - self._prev_right_degree
27         self._prev_left_degree = left_degree
28         self._prev_right_degree = right_degree
29         return delta_left_degree, delta_right_degree
30 class Odom:
31     def __init__(self, robot, wheel_radius, base_length):
32         self._robot = robot
33         self._wheel_radius = wheel_radius
34         self._base_length = base_length
35         self._x = 0
36         self._y = 0
37         self._theta = 0
38     @property
39     def x(self):
40         return self._x
41     @property
42     def y(self):
43         return self._y
44     @property
45     def theta(self):
46         return self._theta
47     def update(self):
48         delta_left_degree, delta_right_degree = self._robot.
49 get_delta_degrees()
50         delta_left_radians = radians(delta_left_degree)
51         delta_right_radians = radians(delta_right_degree)
52         delta_length = (
53             (delta_left_radians + delta_right_radians) * self._wheel_radius
54             * 0.5)
55         delta_theta = (
56             (delta_right_radians - delta_left_radians)
57             * self._wheel_radius
58             / self._base_length)
59         theta = self._theta + delta_theta * 0.5
60         self._x += delta_length * cos(theta)
61         self._y += delta_length * sin(theta)
62         self._theta += delta_theta
63 class P:
64     def __init__(self, linear_speed_coef, angle_speed_coef):
65         self._linear_speed_coef = linear_speed_coef
66         self._angle_speed_coef = angle_speed_coef
67     def calc_powers(self, length_error, theta_error):
68         linear_speed = self._linear_speed_coef * length_error
69         angle_speed = self._angle_speed_coef * theta_error
70         angle_speed = min(MAX_SPEED, max(-MAX_SPEED, angle_speed))
71         if abs(linear_speed) + abs(angle_speed) > MAX_SPEED:
72             linear_speed = copysign(MAX_SPEED - abs(angle_speed),
73 linear_speed)
74         return linear_speed - angle_speed, linear_speed + angle_speed
75 class Tangential:
76     def __init__(self, linear_speed_coef, angle_speed_coef):

```

```

74     self._linear_speed_coef = linear_speed_coef
75     self._angle_speed_coef = angle_speed_coef
76     def calc_powers(self, length_error, theta_error):
77         linear_speed = self._linear_speed_coef * length_error * cos(
theta_error)
78         angle_speed = (
79             self._linear_speed_coef * cos(theta_error) * sin(theta_error)
80             + self._angle_speed_coef * theta_error)
81         angle_speed = min(MAX_SPEED, max(-MAX_SPEED, angle_speed))
82         if abs(linear_speed) + abs(angle_speed) > MAX_SPEED:
83             linear_speed = copysign(MAX_SPEED - abs(angle_speed),
linear_speed)
84             return linear_speed - angle_speed, linear_speed + angle_speed
85 class Planner:
86     def __init__(self, robot, odom):
87         self._robot = robot
88         self._odom = odom
89     @staticmethod
90     def _wrap_angle(angle):
91         while angle > pi:
92             angle -= 2.0 * pi
93         while angle < -pi:
94             angle += 2.0 * pi
95         return angle
96     def move_to_position(self, target, target_length_error, regulator, file)
:
97         target_x, target_y = target
98         length_error = float("inf")
99         start_time = time()
100         while length_error > target_length_error:
101             t = time()
102             self._odom.update()
103             x_error = target_x - self._odom.x
104             y_error = target_y - self._odom.y
105             length_error = (x_error**2 + y_error**2) ** 0.5
106             theta_error = self._wrap_angle(atan2(y_error, x_error) - self.
_odom.theta)
107             powers = regulator.calc_powers(length_error, theta_error)
108             print("odom", self._odom.x, self._odom.y, length_error,
theta_error)
109             self._robot.set_powers(*powers)
110             file.write(
111                 str(t - start_time)
112                 + " "
113                 + str(self._odom.x)
114                 + " "
115                 + str(self._odom.y)
116                 + "\n"
117             )
118             self._robot.set_powers(0, 0)
119     def move_to_positions(self, targets, target_length_error, regulator,
file):
120         for target in targets:
121             self.move_to_position(target, target_length_error, regulator,
file)
122 robot = Robot(LargeMotor(OUTPUT_A), LargeMotor(OUTPUT_B))
123 odom = Odom(robot, 29.0, 120.0)
124 planner = Planner(robot, odom)
125 p = P(0.5, 1)
126 tangential = Tangential(0.5, 1)

```

```

127 path = [
128     (1000, 1000),
129     (-1000, 1000),
130     (-1000, -1000),
131     (1000, -1000),
132     (1000,1000)
133 ]
134 # with open("p10.txt", "w") as file:
135 #     file.write("time x y" + "\n")
136 #     planner.move_to_position((1000, 0), 20.0, p, file)
137 #
138 # with open("p01.txt", "w") as file:
139 #     file.write("time x y" + "\n")
140 #     planner.move_to_position((0, 1000), 20.0, p, file)
141
142 # with open("p-10.txt", "w") as file:
143 #     file.write("time x y" + "\n")
144 #     planner.move_to_position((-1000, 0), 20.0, p, file)
145
146 # with open("p0-1.txt", "w") as file:
147 #     file.write("time x y" + "\n")
148 #     planner.move_to_position((0, -1000), 20.0, p, file)
149
150 # with open("tangential10.txt", "w") as file:
151 #     file.write("time x y" + "\n")
152 #     planner.move_to_position((1000, 0), 20.0, tangential, file)
153
154 # with open("tangential01.txt", "w") as file:
155 #     file.write("time x y" + "\n")
156 #     planner.move_to_position((0, 1000), 20.0, tangential, file)
157
158 # with open("tangential-10.txt", "w") as file:
159 #     file.write("time x y" + "\n")
160 #     planner.move_to_position((-1000, 0), 20.0, tangential, file)
161
162 # with open("tangential0-1.txt", "w") as file:
163 #     file.write("time x y" + "\n")
164 #     planner.move_to_position((0, -1000), 20.0, tangential, file)
165
166 # with open("p_path.txt", "w") as file:
167 #     file.write("time x y" + "\n")
168 #     planner.move_to_positions(path, 25.0, p, file)
169 with open("tangential_path.txt", "w") as file:
170     file.write("time x y" + "\n")
171     planner.move_to_positions(path, 25.0, tangential, file)

```

В Листинг Matlab-скрипта

```

1 %% run_all_little.m
2 % Один файл для 4 лабы: запуск симуляций и построение графиков
3
4 DATA = fullfile('..', 'data');
5 GRAPHICS = fullfile('..', 'graphics');
6
7 %% =====
8 %% ШАГ 1. ПАРАМЕТРЫ
9 %% =====
10 r = 0.029;          % радиус колеса, м
11 B = 0.120;          % база, м
12

```

```

13 K_s = 0.5;          % коэффициенты Прегулятора-
14 K_r = 2;
15
16 K1 = 0.5;          % коэффициенты тангенциального регулятора
17 K2 = 1;
18
19 MAX_U = 50.0;
20
21 assignin('base', 'r', r);
22 assignin('base', 'B', B);
23 assignin('base', 'K_s', K_s);
24 assignin('base', 'K_r', K_r);
25 assignin('base', 'K1', K1);
26 assignin('base', 'K2', K2);
27 assignin('base', 'MAX_U', MAX_U);
28
29 %% =====
30 %% ШАГ 2. СОЗДАНИЕ МОДЕЛЕЙ пропущено( - модели создаются отдельно)
31 %% =====
32 % p_regulator.slx и tangential_regulator.slx должны быть созданы заранее
33
34 %% =====
35 %% ШАГ 3. ЗАГРУЗКА МОДЕЛЕЙ пропущено( - модели загружаются при симуляции)
36 %% =====
37
38 %% =====
39 %% ШАГ 4. СИМУЛЯЦИИ ДЛЯ 4 ТОЧЕК
40 %% =====
41 goals = [ 1  0;
42           0  1;
43          -1  0;
44           0 -1];
45
46 p_keys = {'p10', 'p01', 'pm10', 'p0m1'};
47 t_keys = {'t10', 't01', 'tm10', 't0m1'};
48
49 simP = struct();
50 simT = struct();
51
52 load_system('p_regulator');
53 for i = 1:4
54     set_param('p_regulator/goal_x', 'Value', num2str(goals(i,1)));
55     set_param('p_regulator/goal_y', 'Value', num2str(goals(i,2)));
56     set_param('p_regulator/int_x', 'InitialCondition', '0');
57     set_param('p_regulator/int_y', 'InitialCondition', '0');
58     set_param('p_regulator/int_theta', 'InitialCondition', '0');
59
60     out = sim('p_regulator', 'StopTime', '25', 'ReturnWorkspaceOutputs', 'on');
61
62     st.t = out.x.Time;
63     st.x = out.x.Data;
64     st.y = out.y.Data;
65     st.theta = out.theta.Data;
66     simP.(p_keys{i}) = st;
67 end
68
69 load_system('tangential_regulator');
70 for i = 1:4
71     set_param('tangential_regulator/goal_x', 'Value', num2str(goals(i,1)));

```

```

72     set_param('tangential_regulator/goal_y', 'Value', num2str(goals(i,2)));
73     set_param('tangential_regulator/int_x', 'InitialCondition', '0');
74     set_param('tangential_regulator/int_y', 'InitialCondition', '0');
75     set_param('tangential_regulator/int_theta', 'InitialCondition', '0');
76
77     out = sim('tangential_regulator', 'StopTime', '25', 'ReturnWorkspaceOutputs', 'on');
78
79     st.t = out.x.Time;
80     st.x = out.x.Data;
81     st.y = out.y.Data;
82     st.theta = out.theta.Data;
83     simT.(t_keys{i}) = st;
84 end
85
86 %% =====
87 %% ШАГ 5. СИМУЛЯЦИЯ КВАДРАТА
88 %% =====
89 square_targets = [ ...
90     0  0;
91     1  1;
92    -1  1;
93    -1 -1;
94     1 -1;
95     1  1];
96
97 simSquareP.x = [];
98 simSquareP.y = [];
99 simSquareP.theta = [];
100 simSquareP.t = [];
101
102 x0 = 0; y0 = 0; th0 = 0; time_shift = 0;
103 load_system('p_regulator');
104
105 for k = 2:size(square_targets,1)
106     gx = square_targets(k,1);
107     gy = square_targets(k,2);
108
109     set_param('p_regulator/goal_x', 'Value', num2str(gx));
110     set_param('p_regulator/goal_y', 'Value', num2str(gy));
111     set_param('p_regulator/int_x', 'InitialCondition', num2str(x0));
112     set_param('p_regulator/int_y', 'InitialCondition', num2str(y0));
113     set_param('p_regulator/int_theta', 'InitialCondition', num2str(th0));
114
115     out = sim('p_regulator', 'StopTime', '10', 'ReturnWorkspaceOutputs', 'on');
116
117     x_seg = out.x.Data;
118     y_seg = out.y.Data;
119     th_seg = out.theta.Data;
120     t_seg = out.x.Time + time_shift;
121
122     if ~isempty(simSquareP.x)
123         x_seg = x_seg(2:end);
124         y_seg = y_seg(2:end);
125         th_seg = th_seg(2:end);
126         t_seg = t_seg(2:end);
127     end
128
129     simSquareP.x = [simSquareP.x; x_seg];

```

```

130     simSquareP.y = [simSquareP.y; y_seg];
131     simSquareP.theta = [simSquareP.theta; th_seg];
132     simSquareP.t = [simSquareP.t; t_seg];
133
134     x0 = out.x.Data(end);
135     y0 = out.y.Data(end);
136     th0 = out.theta.Data(end);
137     time_shift = simSquareP.t(end);
138 end
139
140 simSquareT.x = [];
141 simSquareT.y = [];
142 simSquareT.theta = [];
143 simSquareT.t = [];
144
145 x0 = 0; y0 = 0; th0 = 0; time_shift = 0;
146 load_system('tangential_regulator');
147
148 for k = 2:size(square_targets,1)
149     gx = square_targets(k,1);
150     gy = square_targets(k,2);
151
152     set_param('tangential_regulator/goal_x', 'Value', num2str(gx));
153     set_param('tangential_regulator/goal_y', 'Value', num2str(gy));
154     set_param('tangential_regulator/int_x', 'InitialCondition', num2str(x0))
155     ;
156     set_param('tangential_regulator/int_y', 'InitialCondition', num2str(y0))
157     ;
158     set_param('tangential_regulator/int_theta', 'InitialCondition', num2str(
159 th0));
160
161     out = sim('tangential_regulator', 'StopTime', '10', '
162 ReturnWorkspaceOutputs', 'on');
163
164     x_seg = out.x.Data;
165     y_seg = out.y.Data;
166     th_seg = out.theta.Data;
167     t_seg = out.x.Time + time_shift;
168
169     if ~isempty(simSquareT.x)
170         x_seg = x_seg(2:end);
171         y_seg = y_seg(2:end);
172         th_seg = th_seg(2:end);
173         t_seg = t_seg(2:end);
174     end
175
176     simSquareT.x = [simSquareT.x; x_seg];
177     simSquareT.y = [simSquareT.y; y_seg];
178     simSquareT.theta = [simSquareT.theta; th_seg];
179     simSquareT.t = [simSquareT.t; t_seg];
180
181     x0 = out.x.Data(end);
182     y0 = out.y.Data(end);
183     th0 = out.theta.Data(end);
184     time_shift = simSquareT.t(end);
185 end
186
187 %% =====
188 %% ШАГ 6. ПОСТРОЕНИЕ ИТОГОВЫХ ГРАФИКОВ
189 %% =====

```

```

186 p_exp_files = { ...
187     fullfile(DATA, 'p10.txt'), ...
188     fullfile(DATA, 'p01.txt'), ...
189     fullfile(DATA, 'p-10.txt'), ...
190     fullfile(DATA, 'p0-1.txt')};
191
192 t_exp_files = { ...
193     fullfile(DATA, 'tangential10.txt'), ...
194     fullfile(DATA, 'tangential01.txt'), ...
195     fullfile(DATA, 'tangential-10.txt'), ...
196     fullfile(DATA, 'tangential0-1.txt')};
197
198 titles_txt = { ...
199     'Траектория из (0,0) в (1,0)', ...
200     'Траектория из (0,0) в (0,1)', ...
201     'Траектория из (0,0) в (-1,0)', ...
202     'Траектория из (0,0) в (0,-1)'};
203
204 p_sim_keys = {'p10', 'p01', 'pm10', 'p0m1'};
205 t_sim_keys = {'t10', 't01', 'tm10', 't0m1'};
206
207 CPe = [0.00 0.35 0.85];
208 CTe = [0.85 0.10 0.10];
209 CPs = [0.35 0.70 1.00];
210 CTs = [1.00 0.50 0.50];
211
212 %% 4 графика по точкам
213 for i = 1:4
214     dp = readmatrix(p_exp_files{i});
215     dt = readmatrix(t_exp_files{i});
216
217     xp = dp(:,2);
218     yp = dp(:,3);
219
220     xt = dt(:,2);
221     yt = dt(:,3);
222
223     xps = simP.(p_sim_keys{i}).x * 1000;
224     yps = simP.(p_sim_keys{i}).y * 1000;
225
226     xts = simT.(t_sim_keys{i}).x * 1000;
227     yts = simT.(t_sim_keys{i}).y * 1000;
228
229     fig = figure('Visible','off','Position',[100 100 820 680]);
230     hold on;
231
232     plot(xp, yp, '-','Color', CPe, 'LineWidth', 2.1, 'DisplayName', 'P
регулятор-');
233     plot(xt, yt, '-','Color', CTe, 'LineWidth', 2.1, 'DisplayName', '
Тангенциальный');
234     plot(xps, yps, '--','Color', CPs, 'LineWidth', 2.0, 'DisplayName', '
Simulink P');
235     plot(xts, yts, '--','Color', CTs, 'LineWidth', 2.0, 'DisplayName', '
Simulink Tangential');
236
237     plot(0, 0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 7, 'DisplayName',
'Старт');
238
239     xlabel('X, мм');
240     ylabel('Y, мм');

```

```

241     title(titles_txt{i});
242     legend('Location', 'best');
243     grid on;
244     axis equal;
245
246     saveas(fig, fullfile(GRAPHICS, sprintf('point_%d.png', i)));
247     close(fig);
248 end
249
250 %% Общий график
251 fig = figure('Visible','off','Position',[100 100 1050 820]);
252 hold on;
253
254 for i = 1:4
255     dp = readmatrix(p_exp_files{i});
256     dt = readmatrix(t_exp_files{i});
257
258     xp = dp(:,2);
259     yp = dp(:,3);
260
261     xt = dt(:,2);
262     yt = dt(:,3);
263
264     xps = simP.(p_sim_keys{i}).x * 1000;
265     yps = simP.(p_sim_keys{i}).y * 1000;
266
267     xts = simT.(t_sim_keys{i}).x * 1000;
268     yts = simT.(t_sim_keys{i}).y * 1000;
269
270     plot(xp, yp, '-', 'Color', CPe, 'LineWidth', 1.4, 'HandleVisibility',
271          'off');
272     plot(xt, yt, '-', 'Color', CTe, 'LineWidth', 1.4, 'HandleVisibility',
273          'off');
274     plot(xps, yps, '--', 'Color', CPs, 'LineWidth', 1.5, 'HandleVisibility',
275          'off');
276     plot(xts, yts, '--', 'Color', CTs, 'LineWidth', 1.5, 'HandleVisibility',
277          'off');
278 end
279
280 plot(nan,nan,'-', 'Color', CPe, 'LineWidth', 2.0, 'DisplayName', 'Регулятор-');
281 plot(nan,nan,'-', 'Color', CTe, 'LineWidth', 2.0, 'DisplayName', 'Тангенциальный');
282 plot(nan,nan,'--', 'Color', CPs, 'LineWidth', 2.0, 'DisplayName', 'Simulink P');
283 plot(nan,nan,'--', 'Color', CTs, 'LineWidth', 2.0, 'DisplayName', 'Simulink
284       Tangential');
285 plot(0,0,'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 7, 'DisplayName', 'Старт');
286
287 xlabel('X, мм');
288 ylabel('Y, мм');
289 title('Общий график для всех переходов');
290 legend('Location', 'eastoutside');
291 grid on;
292 axis equal;
293
294 saveas(fig, fullfile(GRAPHICS, 'all_points.png'));
295 close(fig);
296
297 %% График по квадрату
298 dp = readmatrix(fullfile(DATA, 'p_path.txt'));
299 dt = readmatrix(fullfile(DATA, 'tangential_path.txt'));
300

```

```

296 xp = dp(:,2);
297 yp = dp(:,3);
298
299 xt = dt(:,2);
300 yt = dt(:,3);
301
302 xSP = simSquareP.x * 1000;
303 ySP = simSquareP.y * 1000;
304
305 xST = simSquareT.x * 1000;
306 yST = simSquareT.y * 1000;
307
308 fig = figure('Visible','off','Position',[100 100 900 760]);
309 hold on;
310
311 plot(xp, yp, '-', 'Color', CPe, 'LineWidth', 2.1, 'DisplayName', 'P
регулятор-');
312 plot(xt, yt, '-', 'Color', CTe, 'LineWidth', 2.1, 'DisplayName', '
Тангенциальный');
313 plot(xSP, ySP, '--', 'Color', CPe, 'LineWidth', 2.0, 'DisplayName', '
Simulink P');
314 plot(xST, yST, '--', 'Color', CTe, 'LineWidth', 2.0, 'DisplayName', '
Simulink Tangential');
315
316 plot(0, 0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 7, 'DisplayName', '
Старт');
317
318 xlabel('X, мм');
319 ylabel('Y, мм');
320 title('Траектория по квадрату');
321 legend('Location', 'best');
322 grid on;
323 axis equal;
324
325 saveas(fig, fullfile(GRAPHICS, 'square_path.png'));
326 close(fig);

```